

Java 8+ Features for Beginners (Tier 1)

A practical guide to the most important Java 8 and later features for interviews. The Stream API was covered in a separate three-part guide, so this guide focuses on everything else.

Quick Map: Which Java Version Has What?

Java has been adding features since Java 8 came out in 2014. Here is a quick map of the most important ones:

- Java 8: Lambda expressions, functional interfaces, method references, default methods, Optional, Stream API, new Date/Time API
- Java 9: Collection factory methods (`List.of`, `Set.of`, `Map.of`), private interface methods
- Java 10: `var` keyword
- Java 11: New String methods (`isBlank`, `strip`, `repeat`, `lines`)
- Java 14: Switch expressions
- Java 15: Text blocks
- Java 16: Records, `Stream.toList()`
- Java 17: Sealed classes (Java 17 is an LTS version, very common in real projects)
- Java 21: Virtual threads, pattern matching for switch (latest LTS)

If you are using Java 8 or higher, you have access to most of the basics in this guide.

What's in Tier 1?

In this guide we cover:

1. Lambda expressions
2. Functional interfaces (Predicate, Function, Consumer, Supplier)
3. Method references
4. Optional
5. Default and static methods in interfaces
6. Collection factory methods (`List.of`, `Set.of`, `Map.of`)

These are the features that show up in almost every Java interview from Java 8 onwards.

1. Lambda Expressions

A lambda is a short way to write a small function. Before Java 8, if you wanted to pass a function to a method, you had to write a lot of code. Lambdas make this clean and short.

The old way (before Java 8)

```
java

List<String> names = Arrays.asList("Bob", "Alice", "Carol");

Collections.sort(names, new Comparator<String>() {
    @Override
    public int compare(String a, String b) {
        return a.compareTo(b);
    }
});
```

That is a lot of code just to say "sort alphabetically".

The new way (Java 8+)

```
java

Collections.sort(names, (a, b) -> a.compareTo(b));
```

One line. Same result.

Lambda syntax

A lambda has three parts:

1. Parameters in parentheses
2. The arrow `->`
3. The body (what to do)

```
java

(int a, int b) -> a + b
```

Different forms

```
java
```

```
// No parameters
() -> System.out.println("Hello")

// One parameter (parentheses are optional)
name -> name.toUpperCase()
(name) -> name.toUpperCase()

// Multiple parameters
(a, b) -> a + b

// Multiple lines (use braces and return)
(a, b) -> {
    int sum = a + b;
    return sum;
}
```

Notice you do not need to write the parameter types most of the time. Java figures them out from the context.

Where can you use lambdas?

Anywhere a functional interface is expected (we explain functional interfaces next).

```
java

// Sort a list
list.sort((a, b) -> a.length() - b.length());

// Loop through a list
list.forEach(item -> System.out.println(item));

// Run code in a thread
new Thread(() -> System.out.println("Running")).start();

// Use with streams
list.stream().filter(n -> n > 5).count();
```

Things to remember

- You can only use lambdas where a functional interface is expected.
- Variables used inside a lambda from outside must be `final` or effectively final (you cannot change them after).

```
java
```

```
int x = 10;
Runnable r = () -> System.out.println(x);    // OK
// x = 20;    // would break the lambda above
```

2. Functional Interfaces

A functional interface is an interface that has exactly one abstract method. That is the only rule. The lambda you write becomes the implementation of that one method.

Custom example

```
java

@FunctionalInterface
interface Greeting {
    void sayHello(String name);
}

Greeting g = name -> System.out.println("Hi " + name);
g.sayHello("Alice");
// Hi Alice
```

The `@FunctionalInterface` annotation is optional but it is good practice. It makes the compiler check that there is only one abstract method. If someone adds another, you get an error right away.

Built-in functional interfaces

Java gives you a set of common ones in `java.util.function`. You will see these everywhere.

Predicate<T> : takes T, returns boolean

For yes or no checks.

```
java

Predicate<Integer> isEven = n -> n % 2 == 0;

isEven.test(4);    // true
isEven.test(7);    // false
```

Used by `filter`, `anyMatch`, `allMatch`, and similar methods.

Function<T, R> : takes T, returns R

For transformations.

```
java
```

```
Function<String, Integer> length = s -> s.length();
```

```
length.apply("hello");    // 5
```

Used by `map`.

Consumer<T> : takes T, returns nothing

For doing something with each item.

```
java
```

```
Consumer<String> printer = s -> System.out.println(s);
```

```
printer.accept("hello");    // prints hello
```

Used by `forEach`.

Supplier<T> : takes nothing, returns T

For creating values.

```
java
```

```
Supplier<String> greeter = () -> "Hello!";
```

```
greeter.get();    // "Hello!"
```

Used by `Stream.generate`, `Optional.orElseGet`, and similar lazy methods.

Quick way to remember the four

Interface	Takes	Returns	Method name
<code>Predicate<T></code>	T	boolean	<code>test</code>
<code>Function<T, R></code>	T	R	<code>apply</code>
<code>Consumer<T></code>	T	nothing	<code>accept</code>
<code>Supplier<T></code>	nothing	T	<code>get</code>

Other useful built-ins

BiFunction, BiPredicate, BiConsumer

Same as the basic ones, but they take two arguments.

java

```
BiFunction<Integer, Integer, Integer> add = (a, b) -> a + b;
add.apply(3, 4);    // 7

BiPredicate<String, Integer> hasLength = (s, n) -> s.length() == n;
hasLength.test("hi", 2);    // true

BiConsumer<String, Integer> printer = (name, age) ->
    System.out.println(name + " is " + age);
printer.accept("Alice", 25);
```

UnaryOperator and BinaryOperator

These are special versions where input and output types are the same.

java

```
// Same as Function<String, String>
UnaryOperator<String> shout = s -> s.toUpperCase() + "!";
shout.apply("hello");    // "HELLO!"

// Same as BiFunction<Integer, Integer, Integer>
BinaryOperator<Integer> sum = (a, b) -> a + b;
sum.apply(3, 4);    // 7
```

Combining predicates

`Predicate` has helper methods to combine checks.

java

```
Predicate<Integer> isPositive = n -> n > 0;
Predicate<Integer> isEven = n -> n % 2 == 0;

Predicate<Integer> isPositiveAndEven = isPositive.and(isEven);
Predicate<Integer> isPositiveOrEven = isPositive.or(isEven);
Predicate<Integer> isNotPositive = isPositive.negate();

isPositiveAndEven.test(4);    // true
isPositiveAndEven.test(-2);    // false
```

This is handy when you want to build complex conditions step by step.

Combining functions

`Function` has `andThen` and `compose` for chaining.

java

```
Function<Integer, Integer> doubled = n -> n * 2;
Function<Integer, Integer> plus10 = n -> n + 10;

// First double, then add 10
Function<Integer, Integer> doubleThen10 = doubled.andThen(plus10);
doubleThen10.apply(5);    // (5 * 2) + 10 = 20

// First add 10, then double
Function<Integer, Integer> add10ThenDouble = doubled.compose(plus10);
add10ThenDouble.apply(5);    // (5 + 10) * 2 = 30
```

`andThen` runs the current function first, then the next one. `compose` runs the next function first, then the current one.

3. Method References

A method reference is a shortcut for a lambda that just calls one method. Instead of writing the full lambda, you write `ClassName::methodName`.

Why use them?

They are shorter and easier to read.

java

```
// Lambda
names.forEach(name -> System.out.println(name));

// Method reference (cleaner)
names.forEach(System.out::println);
```

The four types

1. Static method reference

java

```
Function<Integer, Integer> abs = Math::abs;
abs.apply(-5);    // 5
```

Same as `n -> Math.abs(n)`.

2. Instance method reference (specific object)

java

```
Consumer<String> printer = System.out::println;
printer.accept("hello");
```

Same as `s -> System.out.println(s)`.

3. Instance method reference (any object of a class)

java

```
Function<String, String> upper = String::toUpperCase;
upper.apply("hello"); // "HELLO"
```

Same as `s -> s.toUpperCase()`. Java passes the input as the object on which the method is called.

4. Constructor reference

java

```
Supplier<List<String>> listMaker = ArrayList::new;
List<String> list = listMaker.get(); // new ArrayList
```

Same as `() -> new ArrayList<>()`.

When to use lambdas vs method references

Use method references when your lambda just calls one method.

java

```
// Lambda is fine
names.forEach(name -> System.out.println(name));

// Method reference is cleaner
names.forEach(System.out::println);
```

But if your lambda does anything more, stick with the lambda:

java

```
// Method reference does not work here
names.forEach(name -> System.out.println("Name: " + name));
```

A trick to convert

If your lambda looks like `x -> someMethod(x)`, you can usually replace it with a method

reference.

```
java

list.stream().map(s -> s.toUpperCase());           // lambda
list.stream().map(String::toUpperCase);            // method reference

list.stream().mapToInt(s -> s.length());           // lambda
list.stream().mapToInt(String::length);            // method reference

list.stream().filter(n -> Objects.nonNull(n));     // lambda
list.stream().filter(Objects::nonNull);            // method reference
```

4. Optional

`Optional` is a wrapper class. It says "there might be a value here, or there might not." It helps you avoid `NullPointerException` and makes "could be null" obvious in your code.

The problem it solves

```
java

String name = getNameFromDatabase();    // might be null
System.out.println(name.toUpperCase()); // crash if null!
```

With Optional

```
java

Optional<String> name = getNameFromDatabase();

if (name.isPresent()) {
    System.out.println(name.get().toUpperCase());
}
```

The compiler forces you to think about the case where the value is missing.

Creating an Optional

```
java

Optional<String> a = Optional.of("Alice");           // value, never null
Optional<String> b = Optional.empty();               // no value
Optional<String> c = Optional.ofNullable(maybeNull); // value or empty
```

- Use `of` when you are sure the value is not null.

- Use `ofNullable` when it might be null.
- `Optional.of(null)` will crash. `Optional.ofNullable(null)` returns empty.

Common methods

isPresent and isEmpty

```
java

opt.isPresent();    // true if has value
opt.isEmpty();      // true if no value (Java 11+)
```

get

```
java

String name = opt.get();    // crashes with NoSuchElementException if empty
```

Avoid `get()` if you can. There are better methods below.

ifPresent : do something if value exists

```
java

opt.ifPresent(name -> System.out.println(name));
```

orElse : default if empty

```
java

String name = opt.orElse("Unknown");
```

Returns the value if present, otherwise the default.

orElseGet : lazy default

```
java

String name = opt.orElseGet(() -> computeDefault());
```

Same as `orElse`, but the default is only computed if needed. Use this when computing the default is expensive (like a database call).

orElseThrow : crash if empty

```
java
```

```
// Throws NoSuchElementException
String name = opt.orElseThrow();

// Throws your custom exception
String name = opt.orElseThrow(() -> new RuntimeException("Not found"));
```

map : transform the value

```
java

Optional<String> name = Optional.of("Alice");
Optional<Integer> length = name.map(String::length);
// Optional[5]
```

If the optional is empty, `map` does nothing and returns empty.

filter : keep only if match

```
java

Optional<String> longName = name.filter(s -> s.length() > 5);
// Optional[Alice] becomes Optional.empty() because Alice is only 5 chars
```

ifPresentOrElse (Java 9+)

Two paths in one call.

```
java

opt.ifPresentOrElse(
    name -> System.out.println("Found: " + name),
    () -> System.out.println("Not found")
);
```

or (Java 9+)

Provide a fallback Optional.

```
java

Optional<String> result = opt.or(() -> Optional.of("Default"));
```

If `opt` is empty, the result is `Optional.of("Default")`.

A clean Optional pipeline

```
java
```

```
String result = findUserId(123)                // Optional<User>
    .map(User::getName)                        // Optional<String>
    .filter(name -> !name.isEmpty())           // Optional<String>
    .map(String::toUpperCase)                  // Optional<String>
    .orElse("UNKNOWN");
```

This is clean code. No null checks, no nested ifs.

Important rules for Optional

1. Use `Optional` only as a return type. Do not use it as a field, parameter, or in a collection.
 2. Never call `get()` without checking first. Use `orElse`, `orElseGet`, or `ifPresent` instead.
 3. Do not use `Optional.of(maybeNull)`. Use `ofNullable`.
 4. Do not return `null` from a method that returns `Optional`. Return `Optional.empty()`.
-

5. Default and Static Methods in Interfaces

Before Java 8, interfaces could only have abstract methods (just signatures). Now they can have actual code too.

Default methods

A default method has a body. Classes that implement the interface can use it as is, or override it.

```
java

interface Greeter {
    String getName();

    default String hello() {
        return "Hello, " + getName();
    }
}

class Person implements Greeter {
    public String getName() {
        return "Alice";
    }
    // hello() is inherited automatically
}

Person p = new Person();
p.hello();    // "Hello, Alice"
```

Why are they useful?

To add new methods to an existing interface without breaking all the classes that already implement it.

For example, the `List` interface got a `sort` method as a default method in Java 8. All the old code that implemented `List` did not need to change.

```
java

interface List<E> {
    // ... old methods ...

    default void sort(Comparator<? super E> c) {
        // default implementation here
    }
}
```

Static methods in interfaces

You can also have static methods. These are called using the interface name.

```
java

interface Calculator {
    static int square(int n) {
        return n * n;
    }
}

int result = Calculator.square(5);    // 25
```

These are useful for utility methods that relate to the interface.

Private methods in interfaces (Java 9+)

You can have private methods to share code between default methods.

```
java
```

```

interface Greeter {
    String getName();

    default String hello() {
        return greeting() + ", " + getName();
    }

    default String hi() {
        return greeting() + " " + getName();
    }

    private String greeting() {
        return "Welcome";
    }
}

```

The diamond problem

If a class implements two interfaces with the same default method, you must override it.

```

java

interface A {
    default void hello() {
        System.out.println("A");
    }
}

interface B {
    default void hello() {
        System.out.println("B");
    }
}

class C implements A, B {
    @Override
    public void hello() {
        A.super.hello();    // pick which one to use
    }
}

```

If you do not override, the code will not compile.

When to use default methods vs abstract classes

Use default methods when:

- You want to add a new method to an existing interface
- You want to provide a sensible default that most classes can reuse
- The behavior does not depend on a lot of state

Use abstract classes when:

- You need to share fields (state) between subclasses
 - You need a constructor with logic
 - You want full inheritance with protected methods
-

6. Collection Factory Methods (Java 9+)

A quick way to create small collections that cannot be changed.

List.of

```
java

List<Integer> nums = List.of(1, 2, 3, 4, 5);
List<String> names = List.of("Alice", "Bob");
```

Cleaner than the old way:

```
java

// Old way (still works)
List<Integer> nums = new ArrayList<>();
nums.add(1);
nums.add(2);
nums.add(3);

// Or with Arrays.asList
List<Integer> nums = Arrays.asList(1, 2, 3);
```

Set.of

```
java

Set<Integer> nums = Set.of(1, 2, 3);
Set<String> names = Set.of("Alice", "Bob");
```

If you pass duplicates, it throws an error:

```
java
```

```
Set.of(1, 2, 2);    // crash: IllegalArgumentException
```

Map.of

```
java
Map<String, Integer> ages = Map.of(
    "Alice", 25,
    "Bob", 30
);
```

`Map.of` takes alternating keys and values, up to 10 pairs.

Map.ofEntries (for more pairs)

```
java
Map<String, Integer> ages = Map.ofEntries(
    Map.entry("Alice", 25),
    Map.entry("Bob", 30),
    Map.entry("Carol", 28),
    Map.entry("Dan", 22)
);
```

Use this when you have more than 10 entries, or when you just like the cleaner syntax.

Important: these are immutable

You cannot change them after creation.

```
java
List<Integer> nums = List.of(1, 2, 3);
nums.add(4);    // crash: UnsupportedOperationException
nums.set(0, 9); // crash too
```

If you need a list you can change, wrap it:

```
java
List<Integer> nums = new ArrayList<>(List.of(1, 2, 3));
nums.add(4);    // now this works
```

Important: no null values

You cannot put `null` in any of these.

```
java
```

```
List.of("Alice", null);    // crash: NullPointerException
Set.of(1, null);          // crash
Map.of("key", null);      // crash
```

If you need null values, use the old `HashMap` or `ArrayList`.

copyOf methods (Java 10+)

Make an immutable copy of an existing collection.

```
java

List<Integer> mutable = new ArrayList<>(List.of(1, 2, 3));
mutable.add(4);

List<Integer> copy = List.copyOf(mutable);    // immutable copy
// [1, 2, 3, 4]

mutable.add(5);    // does not affect copy
```

This is great for protecting your data when returning from methods.

Putting It All Together

Real examples that mix these features.

Example 1: Sort a list of people by age

```
java

record Person(String name, int age) {}

List<Person> people = new ArrayList<>(List.of(
    new Person("Carol", 30),
    new Person("Alice", 25),
    new Person("Bob", 28)
));

people.sort(Comparator.comparingInt(Person::age));
```

`Person::age` is a method reference. `Comparator.comparingInt` builds a comparator from it.

Example 2: Find a user safely

```
java
```

```
Optional<Person> found = people.stream()
    .filter(p -> p.name().equals("Alice"))
    .findFirst();

String name = found.map(Person::name).orElse("Unknown");
```

Example 3: Build a configuration map

```
java

Map<String, String> config = Map.of(
    "host", "localhost",
    "port", "8080",
    "timeout", "30"
);

String host = config.getOrDefault("host", "127.0.0.1");
```

Example 4: Combine predicates for filtering

```
java

Predicate<Person> isAdult = p -> p.age() >= 18;
Predicate<Person> hasShortName = p -> p.name().length() <= 4;

List<Person> result = people.stream()
    .filter(isAdult.and(hasShortName))
    .collect(Collectors.toList());
```

Example 5: Process Optional cleanly

```
java

String result = Optional.ofNullable(getRawValue())
    .map(String::trim)
    .filter(s -> !s.isEmpty())
    .map(String::toUpperCase)
    .orElse("DEFAULT");
```

If any step gives empty or null, you get "DEFAULT". No null checks needed.

Example 6: Functional interface with method composition

```
java
```

```
Function<String, String> trim = String::trim;
Function<String, String> upper = String::toUpperCase;
Function<String, Integer> length = String::length;

Function<String, Integer> pipeline = trim.andThen(upper).andThen(length);

pipeline.apply("  hello  "); // 5
```

Common Mistakes

Mistake 1: Using Optional.get() without checking

```
java

Optional<String> opt = findName();
String name = opt.get(); // crash if empty!
```

Use `orElse`, `orElseGet`, or `ifPresent` instead.

Mistake 2: Using Optional as a field

```
java

class Person {
    private Optional<String> nickname; // do not do this
}
```

`Optional` is meant for return values from methods, not for fields. Use a regular field that can be null, or refactor.

Mistake 3: Returning null from a method that returns Optional

```
java

public Optional<User> findUser(int id) {
    if (notFound) {
        return null; // never do this
    }
    return Optional.of(user);
}
```

Return `Optional.empty()` instead. The whole point of Optional is to avoid null.

Mistake 4: Trying to add to an immutable list

```
java
List<Integer> nums = List.of(1, 2, 3);
nums.add(4);    // crash
```

Either wrap in `new ArrayList<>(...)` or use a mutable list from the start.

Mistake 5: Modifying captured variables in lambdas

```
java
int counter = 0;
list.forEach(item -> counter++);    // does not compile
```

Lambdas can only use final or effectively final variables. Use a different approach like `count()` on a stream.

Mistake 6: Using lambdas where method references would be cleaner

```
java
// Verbose
list.stream().map(s -> s.toUpperCase()).forEach(s -> System.out.println(s));

// Clean
list.stream().map(String::toUpperCase).forEach(System.out::println);
```

Mistake 7: Putting null in factory methods

```
java
List.of("Alice", null);    // crash
Map.of("key", null);    // crash
```

Use `new ArrayList<>(...)` or `new HashMap<>()` if you need nulls.

Mistake 8: Forgetting to override conflicting default methods

```
java
class C implements A, B {    // both have default hello()
    // does not compile if you do not override hello()
}
```

Quick Cheat Sheet

You want to...	Use
Pass a function as an argument	Lambda expression
Define your own functional interface	<code>@FunctionalInterface</code>
Take T, return boolean	<code>Predicate<T></code>
Take T, return R	<code>Function<T, R></code>
Take T, return nothing	<code>Consumer<T></code>
Take nothing, return T	<code>Supplier<T></code>
Two arguments instead of one	<code>BiFunction</code> , <code>BiPredicate</code> , <code>BiConsumer</code>
Combine two predicates	<code>predA.and(predB)</code> , <code>.or(...)</code> , <code>.negate()</code>
Chain two functions	<code>funcA.andThen(funcB)</code> or <code>.compose(...)</code>
Shortcut for <code>x -> ClassName.method(x)</code>	Method reference: <code>ClassName::method</code>
Wrap "maybe null" value	<code>Optional<T></code>
Get value with default	<code>opt.orElse(default)</code>
Get value with lazy default	<code>opt.orElseGet(() -> compute())</code>
Get value or throw	<code>opt.orElseThrow(...)</code>
Add method to existing interface	Default method
Utility method on interface	Static method on interface
Quick immutable list	<code>List.of(...)</code>
Quick immutable map	<code>Map.of(...)</code> or <code>Map.ofEntries(...)</code>
Make a defensive copy	<code>List.copyOf(...)</code> , <code>Map.copyOf(...)</code>

Final Tips

1. Lambda expressions and functional interfaces are the foundation. Once you understand these, everything else clicks.

2. Memorize the four main functional interfaces: `Predicate`, `Function`, `Consumer`, `Supplier`. These cover most cases.
3. Use method references whenever your lambda just calls one method. Cleaner code.
4. `Optional` is your friend, but only for return types. Do not overuse it.
5. Always prefer `orElseGet` over `orElse` when the default is expensive to compute.
6. The collection factory methods (`List.of`, `Map.of`) are great for constants and test data, but remember they are immutable.
7. Practice writing small lambdas every day. Soon they will feel as natural as `if` and `for`.

These features changed how Java code is written. Modern Java code looks very different from Java 7 code, and these are the building blocks. Master these before moving on to streams, records, sealed classes, and other newer features.

When you are ready, the next tier covers things like records, switch expressions, text blocks, the new Date/Time API, and more.